

INGENIERÍA DE SERVICIOS DE TELECOMUNICACIÓN

CURSO 2025/26

PRÁCTICA 2: DESARROLLO DE SERVICIOS REST

RESUMEN DE LA PRÁCTICA

En esta práctica se aplicarán los conocimientos adquiridos en el tema 3 para el desarrollo de servicios REST. Las sesiones de la práctica están enfocadas a implementar una aplicación que simule un servicio de control de vehículos de un parking. El servicio que se pondrá en marcha debe ser capaz de:

- 1) Almacenar en una base de datos la matrícula del vehículo que ha accedido al aparcamiento. Mantener un histórico de uso de entradas y salidas de vehículos.
- 2) Dada la matrícula de un vehículo, calcular el tiempo que ha utilizado el servicio y cálculo del coste del aparcamiento.

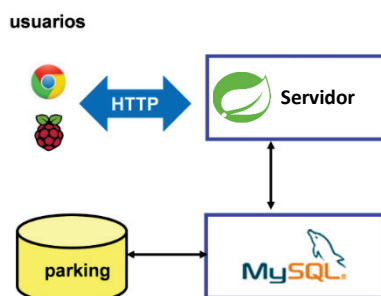


Figura 1. Servicio de parking.

EVALUACIÓN DE LA PRÁCTICA

Esta práctica tendrá un peso del **30%** sobre la nota de prácticas en la modalidad de Evaluación Global.

La práctica se evaluará siguiendo la rúbrica asociada a la práctica 2 y disponible en PLATEA. Esta rúbrica pretende evaluar los resultados de aprendizaje de la práctica en dos fases. En una primera fase se usarán criterios y aspectos relacionados con la entrega, desarrollo y defensa oral del trabajo llevado a cabo (sesiones 1-2). En una segunda fase, en la sesión 3, habrá una evaluación individual en el laboratorio donde se pedirá implementar servicios REST según unas determinadas especificaciones. Para esta prueba se podrá disponer de los códigos desarrollados anteriormente, así como los apuntes de clase.

DESCRIPCIÓN DE LAS SESIONES DE LA PRÁCTICA

La práctica se divide en las siguientes sesiones:

SESIÓN 1. SERVICIO REST DE INSERCIÓN DE MATRÍCULAS.

Implementación del servicio

En esta sesión el alumno va a familiarizarse con la creación de servicios REST destinados a insertar datos que provienen de dispositivos clientes. Más en concreto, se implementará un servicio REST encargado de recoger los valores de las matrículas leídas y de almacenar los registros en una base de datos.

Para implementar el servicio, se recomienda que exista en la base de datos una tabla con los siguientes atributos:

- **id**: Clave primaria numérica y autoincrementada para identificar cada registro.
- **entrada**: Booleano que identifica si el coche ha entrado o salido del parking.
- **matricula**: String con la matrícula del vehículo.
- **hora**: Fecha del registro, de tipo `TIMESTAMP DEFAULT CURRENT_TIMESTAMP`.

id	entrada	matricula	hora

Tabla 1. Atributos de la tabla.

Si se va a trabajar con JPA, el atributo *hora* del DTO se ha de marcar con `@CreationTimestamp(source=SourceType.DB)`, para indicar que el valor lo gestiona la base de datos.

En cuanto a los requisitos del servicio REST, se pide que la petición HTTP sea de tipo POST, y se esperan recibir dos parámetros en los datos de la petición:

- **entrada**. El parámetro 'entrada' lo usaremos para saber si estamos mandando los datos desde la barrera de entrada (true) o de salida (false).
- **matricula**. Se tratará de un String con los caracteres de la matrícula.

NOTA: Los nombres de los ficheros del controlador y de las urls de los servicios deben incorporar algún identificador de la cuenta TIC de estudiante como parte del nombre.

Para testear el servicio implementado, se recomienda usar las herramientas cURL y Postman, siguiendo los ejemplos del Tema 3.

SESIÓN 2. CÁLCULO DEL COSTE DE APARCAMIENTO.

Implementación del servicio

En esta sesión se va a completar la anterior con el cálculo del coste realizado por el usuario, también mediante un servicio REST. El coste asociado a la estancia de un vehículo se calculará según la fórmula:

$$\text{Coste} = (\text{Horasalida} - \text{Horaentrada}) * \text{Tarifa}$$

De esta manera, se calcula como la diferencia entre el tiempo de salida y entrada multiplicada por una tarifa. Para simplificar, el valor de la tarifa se supone fijo, y se incorpora como una variable dentro del código. Teniendo esto en cuenta, se debe proporcionar un servicio REST que devuelva el coste dada una matrícula. La petición HTTP será de tipo GET y la **matrícula será una parte variable de la url del servicio**. Por otra parte, el resultado de la consulta se incluirá en el **cuerpo de la respuesta HTTP como un número** (p.e. tipo double) serializado en json.

De cara a facilitar la implementación, para manejar valores almacenados como Timestamp en una base de datos SQL, existen diversos métodos tales como getTime() de la clase Timestamp, que permite obtener una representación del tiempo almacenado en milisegundos. Así, el siguiente código de ejemplo calcula la diferencia en segundos entre dos variables de tipo Timestamp:

```
Timestamp tiemposalida = ..... //Se obtiene el tiempo de salida
Timestamp tiempoentrada = ..... //Se obtiene el tiempo de entrada
System.out.println("Segundos de estancia: "+((tiemposalida.getTime()-
tiempoentrada.getTime())/1000) );
```

NOTA: Los nombres de los ficheros del controlador y de las urls de los servicios deben incorporar algún identificador de la cuenta TIC de estudiante como parte del nombre.

Para probar este servicio se recomienda usar las herramientas cURL y Postman, aunque al ser de tipo GET también se puede testear escribiendo la URL en un navegador web.

Lectura de la matrícula con Raspberry Pi y OCR

En este apartado se va a utilizar un dispositivo Raspberry Pi 3 que, mediante una cámara web y un software OCR, simulará la lectura de matrículas de coches. Para ello se ejecutará el código en Python 'leematricula.py', que existe ya en la imagen instalada en los dispositivos Raspberry Pi. Se debe comprobar el correcto funcionamiento de la aplicación, simulando el reconocimiento de matrículas y observando el resultado obtenido. El código se ejecuta escribiendo en la línea de comandos: "python leematricula.py"

Se completará la aplicación con el código correcto para enviar los datos de las matrículas de los vehículos al servicio REST encargado de almacenarlas en la base de datos. A continuación, se muestra un código ejemplo para implementar el cliente que consume un servicio HTTP POST, y con el envío de los datos serializados en JSON. Asimismo, este código muestra en pantalla la respuesta del servidor, tanto el status de la respuesta como los datos recibidos (también en JSON).

```
import requests
import json

params = {'entrada': 'true', 'matricula': words}
data_json = json.dumps(params)
headers = {"Content-type": "application/json"}
url = 'http://192.168.0.101:8080/parking/registroMatricula'
print "Contactando con servidor..."
response = requests.post(url, data=data_json, headers=headers)
print "Respuesta de servidor: "
print response
print response.json()
```

Para facilitar el uso del cliente, existe un fichero llamado leematriculacliente.py con este código añadido. De esta manera tan solo se debe cambiar la url para usar su servicio, y el valor del campo entrada para simular si la lectura se produce en la barrera de entrada o en la de salida.

SESIÓN 3. PRUEBA DE EVALUACIÓN.

El día de la prueba de evaluación se entregará un guión con una serie de especificaciones para implementar servicios REST. Al final de esta prueba se deberán entregar los códigos desarrollados a través de la plataforma PLATEA.